

Q36 – Smart Agriculture Irrigation Optimization Using Greedy Scheduling

1. Title

Smart Agriculture Irrigation Optimization Using Greedy Scheduling

2. Aim

To design and implement a **Greedy Scheduling Algorithm** for optimizing agricultural irrigation by considering soil moisture, crop priority, water requirements, deadlines, and weather conditions.

3. Problem Statement

Agricultural irrigation requires careful management of limited water resources. Providing too much water can cause wastage, while insufficient irrigation can negatively affect crop growth.

The objective is to develop an irrigation scheduling system that prioritizes fields based on their urgency and environmental conditions. The system should consider:

- Soil moisture level
- Water requirement of each crop
- Crop priority
- Irrigation deadline
- Weather conditions
- Available water

A greedy approach is used to schedule the most important irrigation tasks first while conserving available water.

4. Objectives

The main objectives are:

1. To optimize irrigation scheduling using a greedy approach.
 2. To prioritize fields with low soil moisture.
 3. To consider crop importance and irrigation deadlines.
 4. To avoid irrigation when rainfall is expected.
 5. To utilize the available water efficiently.
 6. To analyze the time and space complexity of the algorithm.
 7. To understand how the solution can be adapted for real-time agriculture.
-

5. Input Description

Each agricultural field contains the following information:

Parameter	Description
Field	Name/identifier of the field
Soil Moisture	Current moisture percentage
Water Required	Amount of water required in litres
Crop Priority	Importance of the crop
Deadline	Latest preferred irrigation slot
Weather	Current/predicted weather condition

The system also takes the **total available water** as input.

Sample Input

Field	Soil Moisture	Water Required	Crop Priority	Deadline	Weather
A	25%	40 L	5	2	Sunny
B	45%	30 L	3	3	Rain
C	20%	50 L	5	1	Sunny
D	35%	25 L	4	2	Cloudy
E	50%	20 L	2	4	Sunny

Available Water = 120 L

6. Greedy Approach

The system assigns a priority score to each field.

The score considers:

- Moisture deficit
- Crop priority
- Deadline urgency
- Weather conditions

The basic priority calculation is:

Priority Score = Moisture Deficit $\times$ 0.1 + Crop Priority + Deadline Score – Weather Penalty

Fields are then sorted in **descending order of priority**.

The field with the highest priority is considered first. If enough water is available, it is scheduled for irrigation. If rainfall is expected, the field is skipped to avoid unnecessary water usage.

This process continues until all fields have been considered or the available water becomes insufficient.

7. Algorithm

1. Start.
2. Read field and irrigation information.
3. Calculate the moisture deficit for every field.
4. Calculate the deadline urgency.
5. Apply a weather penalty when rain is expected.
6. Calculate the priority score.
7. Sort all fields by priority score in descending order.
8. Set the available water.
9. Select the highest-priority field.
10. If rain is expected, skip the field.
11. Otherwise, if sufficient water is available, schedule irrigation.
12. Deduct the required water from the remaining water.
13. Repeat until all fields are processed.
14. Display the optimized irrigation schedule.
15. Display water usage and remaining water.
16. Stop.

8. Pseudocode

START

Read field information

Read available water

FOR each field

 Calculate moisture deficit

 Calculate deadline urgency

 IF weather is Rain

 Apply weather penalty

 END IF

 Calculate priority score

END FOR

Sort fields by priority score in descending order

FOR each field in sorted order

 IF weather is Rain

 Skip field

 Continue

 END IF

 IF required water $\leq$ remaining water

 Schedule field

 Reduce remaining water

 ELSE

 Skip field

 END IF

END FOR

Display irrigation schedule

Display total water used

Display remaining water

Display skipped fields

STOP

9. Source Code

The implementation is written in **Python** and uses lists, dictionaries, sorting, and a greedy scheduling strategy.

```
1 # Q36 - Smart Agriculture Irrigation Optimization
2 # Greedy Scheduling Algorithm
3
4 def calculate_priority(field):
5     """
6     Calculate priority score based on:
7     - Soil moisture deficit
8     - Crop priority
9     - Deadline urgency
10    - Weather condition
11    """
12
13    moisture_deficit = 100 - field["soil_moisture"]
14
15    # Higher urgency for earlier deadlines
16    deadline_score = 6 - field["deadline"]
17
18    # Weather adjustment
19    if field["weather"].lower() == "rain":
20        weather_penalty = 5
21    elif field["weather"].lower() == "cloudy":
22        weather_penalty = 2
23    else:
24        weather_penalty = 0
25
26    score = (
27        moisture_deficit * 0.1
28        + field["crop_priority"]
29        + deadline_score
30        - weather_penalty
31    )
32
```

Figure 1: Python source code for the irrigation optimization system.

10. Execution and Output

The program was executed using the given sample agricultural field data.

The algorithm calculates a priority score for each field and schedules the fields with the highest priority while considering weather and available water.

```
=====
SMART AGRICULTURE IRRIGATION OPTIMIZER
=====

Available Water : 120 L

OPTIMIZED IRRIGATION SCHEDULE
-----
Slot  Field      Moisture  Water(L)  Priority
-----
1     Field C    20%      50        18.0
2     Field A    25%      40        16.5
3     Field D    35%      25        12.5
-----

Total Water Used   : 115 L
Remaining Water    : 5 L
Fields Scheduled    : 3
Fields Not Scheduled : 2
Water Utilization   : 95.83%
-----

SKIPPED FIELDS
-----
Field E  Priority: 9.0  Reason: Insufficient water
Field B  Priority: 6.5  Reason: Rain expected
-----

=== Code Execution Successful ===
```

Figure 2: Program execution and optimized irrigation schedule.

11. Result Analysis

The program produces an optimized irrigation schedule based on the calculated priority scores.

Fields with lower soil moisture receive higher priority because they have a greater need for irrigation. Crops with higher importance also receive preference.

Fields where rainfall is expected are skipped because irrigating them would result in unnecessary water consumption.

The available water is allocated to the most important fields first.

Performance Measures

The program reports:

- Total water available
- Total water used
- Remaining water
- Number of fields scheduled
- Number of fields skipped
- Water utilization percentage

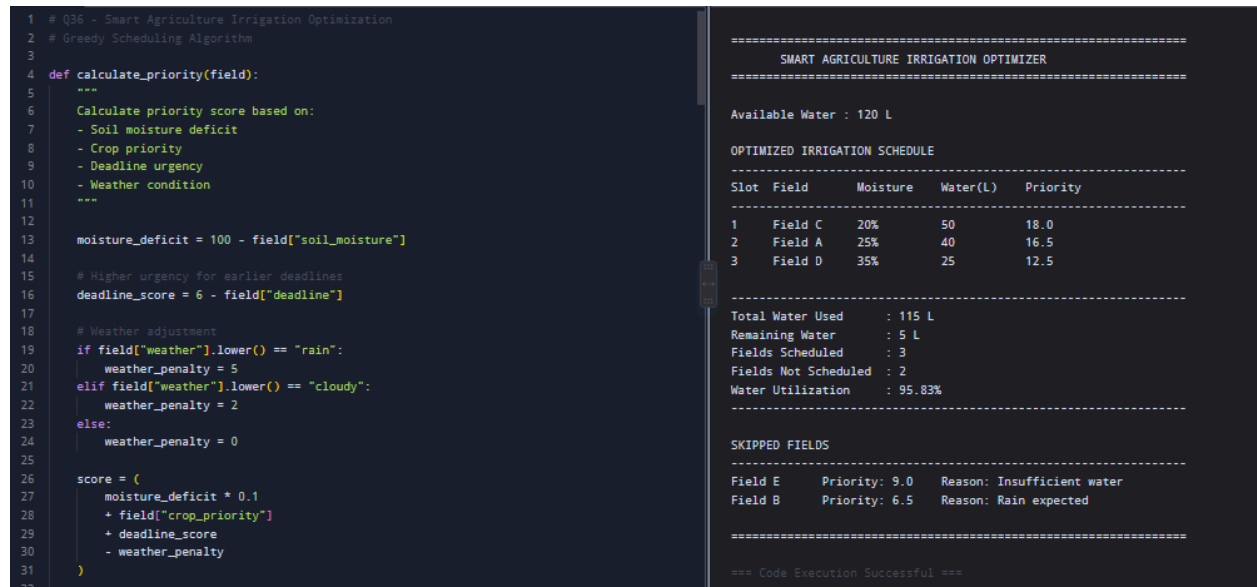


Figure 3: Water utilization and scheduling analysis.

12. Time Complexity Analysis

Let n be the number of agricultural fields.

Priority Calculation

The priority score is calculated once for each field.

Time Complexity = $O(n)$

Sorting

The fields are sorted according to their priority scores.

Time Complexity = $O(n \log n)$

Scheduling

Each field is examined once.

Time Complexity = $O(n)$

Therefore, the overall complexity is:

$$O(n) + O(n \log n) + O(n)$$

Overall Time Complexity

$$O(n \log n)$$

The sorting operation is the dominant operation.

13. Space Complexity

The program stores information about every agricultural field and its calculated priority.

Therefore:

$$\text{Space Complexity} = O(n)$$

where n is the number of fields.

14. Advantages

- Simple and easy to implement.
 - Efficient for large numbers of fields.
 - Reduces unnecessary irrigation.
 - Gives priority to water-stressed crops.
 - Considers weather conditions.
 - Makes effective use of limited water.
 - Can be extended to real-time sensor-based systems.
-

15. Limitations

The greedy algorithm makes decisions based on the current priority scores. It does not guarantee a globally optimal schedule for every possible irrigation problem.

Other limitations include:

- Weather predictions may be inaccurate.
- Soil moisture sensors may provide noisy readings.
- Water availability can change unexpectedly.

- The current implementation uses predefined input data.
 - Irrigation duration and multiple irrigation zones are not considered.
-

16. Real-Time Adaptability

The proposed system can be extended into a real-time smart agriculture system.

Soil-moisture sensors can continuously provide moisture readings from different fields. Weather APIs can provide rainfall forecasts and temperature information.

Whenever new data is received, the system can:

1. Update soil moisture values.
2. Update weather conditions.
3. Recalculate priority scores.
4. Re-sort irrigation tasks.
5. Generate a new schedule.
6. Adjust irrigation according to current conditions.

This allows the irrigation system to respond dynamically to changing agricultural conditions.

17. Efficiency and Scalability

The algorithm has a time complexity of $O(n \log n)$, making it suitable for handling a large number of agricultural fields.

For a large farm, fields can be divided into irrigation zones. Each zone can maintain its own irrigation schedule while sharing information about total water availability.

The same approach can also be integrated with IoT sensors, weather services, and automated irrigation controllers.

18. Sustainable Agriculture Interpretation

Efficient irrigation is important for sustainable agriculture because freshwater is a limited resource.

The proposed system reduces water wastage by:

- Avoiding irrigation when rain is expected.
- Prioritizing fields with low soil moisture.
- Allocating limited water to high-priority crops.
- Avoiding unnecessary irrigation of healthy fields.

Thus, the system supports **water conservation, improved resource utilization, and sustainable crop management**.

19. Conclusion

The Smart Agriculture Irrigation Optimization System successfully demonstrates the application of a **Greedy Scheduling Algorithm** to agricultural irrigation.

By considering soil moisture, crop priority, deadlines, weather conditions, and available water, the system generates an efficient irrigation schedule.

The algorithm requires **$O(n \log n)$ time** and **$O(n)$ space**, making it computationally efficient and scalable.

With the integration of real-time sensors and weather data, the proposed system can be further developed into an intelligent automated irrigation solution for sustainable agriculture.